

Optimizing TinyML for Indoor Localization on Resource-Constrained Devices

Mueid Islam Arian, Mahamudul Hasan, Nafiz Un-Nabi Ishan
Department of Electrical and Computer Engineering, North South University
{mueid.arian, mahamudul.hasan, nafiz.ishan}@northsouth.edu

Abstract—Indoor localization has become a key enabler for applications in smart buildings, asset tracking, and context-aware systems. One of the problems with modern state of the art models is how resource heavy and power consuming they are making them quite difficult to use in resource constrained environments. Deploying deep learning models for accurate localization on resource limited edge devices to this day remains a challenge. In this project, we propose an optimized TinyML framework for indoor localization on edge devices by using model quantization and knowledge distillation to lightweight the architecture for on device indoor localization. We try to minimize the model size, power consumption and latency while keeping the accuracy as close as to the original models, particularly for healthcare applications like patient monitoring.

Index Terms—TinyML, Indoor Localization, Edge Devices, Model Optimization

I. INTRODUCTION

We live in a digital age where privacy and security can often quite easily be threatened especially when almost everything we use in our daily lives relies on digital data which is processed in cloud. In healthcare or in smart-homes where most of it depends on digital data to optimize patient outcomes the ability to accurately track and monitor individuals within indoor environments becomes crucial[1]. However, in third world countries where resources are scarce, security and privacy is often questionable, on device optimized models can mitigate that problem by processing data locally. Indoor localization is essential for applications in patient tracking, smart buildings, and more. Traditional machine learning models require substantial computational resources, limiting their deployment on microcontrollers. Generally, accurate indoor localization rely on large machine learning models that are deployed in a centralized remote server that uses data collected through battery powered edge or IoT devices such as smart watches. This means the raw data is communicated off device which leads to privacy concerns and higher costs. In recent years the advancement in TinyML has made it possible to deploy machine learning algorithms on small, resource constrained edge devices. This allows on device localization to be not only effective but also provide better privacy and security as sensitive data are processed locally rather than transmitting to remote servers. However this field still remains challenging as low power microcontroller units(MCUs) have very limited memory, power and battery life. We try to address these challenges by deploying a lightweight TinyML-based model optimized for such constrained environments.

II. RELATED WORK

One study shows accurate discovery of factory conditions to insure sustainable agrarian productivity and food security by developing a featherlight, real-time factory complaint discovery model using TinyML, specifically targeting apple and mango leaves[2]. Another study addresses the challenge non-invasive blood glucose position(BGL) monitoring for diabetes operation, aiming to replace traditional cutlet poking styles with a comfortable, nonstop, and accurate volition[3]. Their result, a refined segmentation system bettered model delicacy and computational effectiveness. Embedded deployment demonstrated practical connection for nonstop monitoring. A lot of these work are done by model contraction ways like quantization, pruning, and knowledge distillation. These are designed to reduce model size, however each of these come with their own limitations. The summarized comparison of these model compression is presented in Table I[1]. Quantization and knowledge distillation are among the most popular TinyML methods for model compression[4]. Quantization reduces model size and computational cargo by dwindling the perfection of the figures representing the model's parameters, which can incontinently reduce the model's size by half[5]. It's straightforward operation allows for easy implementation, especially when deploying formerly small and effective models on resource- constrained devices.

TABLE I
COMPARISON OF MODEL COMPRESSION TECHNIQUES FOR TINYML

Characteristic	Quant.	KD	Prune	Low-rank
Memory reduction	High	High	High	Moderate
Hardware req.	None	None	Require	None
Ease of impl.	Easy	Mod.	Mod.	Complex
Perf. preserved	Mod.	High	Mod.	Low
TinyML suitability	High	High	Low	Low

III. PROPOSED SOLUTION AND METHODOLOGY

We plan to deploy a TinyML-based indoor localization system using publicly available datasets. The system is designed to run efficiently on resource-constrained edge devices such as the ESP32 or Arduino Nano 33 BLE Sense. The methodology consists of three main phases: *dataset preprocessing*, *model training and optimization*, and *on-device deployment and inference*.

A. Dataset Preprocessing

Publicly available datasets, such as the UJIndoorLoc dataset, provide Wi-Fi fingerprinting data in the form of received signal strength indicator (RSSI) values from multiple access points at known indoor locations. The preprocessing steps include:

- **Data Cleaning:** Remove missing or corrupted RSSI measurements to ensure consistency.
- **Normalization:** Standardize RSSI values to reduce the effect of signal variation across different devices or environments.
- **Feature Construction:** Aggregate RSSI values into feature vectors representing each location sample. Each vector corresponds to a specific room or coordinate in the building.
- **Dataset Splitting:** Divide the dataset into training, validation, and test sets. Typically, 70% of the samples are used for training, 15% for validation, and 15% for testing.

B. Model Training and Optimization

The system employs a lightweight neural network optimized for TinyML deployment. Two main techniques are applied to reduce model size and computational requirements:

1) *Knowledge Distillation (KD)*: A large teacher model is trained on the preprocessed dataset to achieve high localization accuracy. A smaller student model is then trained to replicate the teacher model's output distribution. This process allows the student model to retain most of the performance of the larger model while being significantly smaller and more suitable for edge deployment.

2) *Quantization*: The trained student model is quantized, converting 32-bit floating-point weights to 8-bit integers. Quantization reduces memory usage and improves inference speed on microcontrollers without substantial accuracy loss.

3) *Optional Pruning*: Lightweight pruning may be applied to remove less important connections in the student model. This step is optional and depends on the memory constraints of the target edge device.

C. On-Device Deployment and Inference

The optimized model is deployed on the edge device. For each input feature vector derived from the dataset:

- The model performs inference directly on-device to predict the corresponding indoor location.
- Predictions can be output to a simple interface, such as an LED indicator, a small display, or a serial connection for logging.
- The entire inference process is designed to be real-time, with latency below 100 ms and memory footprint below 100 KB, ensuring compatibility with low-power microcontrollers.

D. Evaluation Metrics

The system's performance is evaluated using standard metrics for indoor localization:

- **Accuracy:** Percentage of correctly predicted locations.

TABLE II
HARDWARE COMPONENTS FOR TINYML INDOOR LOCALIZATION

Component	Purpose	Example Model
Microcontroller	Model deployment	ESP32 / Nano 33 BLE
BLE/WiFi Source	Signal source	BLE Beacon / Router
Laptop/PC	Model training	Any 8GB RAM system
Power Supply	Portable testing	Power bank / USB
Tools	Wiring/setup	Breadboard, cables

TABLE III
SOFTWARE STACK FOR TINYML INDOOR LOCALIZATION

Task	Tools / Libraries	Purpose
Data Collection	ESP-IDF / Arduino / Python	Gather RSSI signals
Model Training	TensorFlow Lite / PyTorch	Build and train models
Optimization	Quantization, KD, Pruning	Reduce model size
Deployment	TFLite Micro / Edge Impulse	On-device inference
Visualization	Matplotlib / Excel	Accuracy analysis

- **Memory Footprint:** Total memory used by the quantized student model.
- **Inference Latency:** Time taken for a single prediction on the edge device.
- **Energy Consumption:** Approximate power usage per inference, measured using device specifications.

This methodology allows for a reproducible, resource-efficient, and fully on-device indoor localization system, making it suitable for low-cost and resource-constrained environments.

IV. EVALUATION PLAN AND EXPECTED RESULTS

The evaluation plan focuses on assessing the performance, efficiency, and feasibility of the proposed TinyML-based indoor localization system on resource-constrained edge devices. The evaluation uses publicly available datasets, such as UJIndoorLoc, and involves both offline analysis and on-device testing.

A. Dataset-Based Evaluation

The preprocessed dataset is divided into training, validation, and test sets (e.g., 70% / 15% / 15%). The test set contains previously unseen samples to evaluate the generalization performance of the model. Key evaluation steps include:

- Measuring localization accuracy at the room or grid level.
- Recording inference latency for each sample on the target edge device.
- Tracking memory footprint and RAM usage of the deployed model.
- Estimating energy consumption per inference to assess battery-powered feasibility.

B. Comparative Evaluation

The system performance is compared across different model optimization strategies:

- **Baseline Model:** Full-sized neural network trained on the dataset.

- **Quantized Model:** Student model with 8-bit quantization.
- **Knowledge Distilled Model:** Student model trained via knowledge distillation (KD).
- **Quantized + KD Model:** Student model with both KD and quantization applied.

Trade-offs between accuracy, memory usage, and inference latency are analyzed for each configuration.

C. Edge Device Testing

The optimized student model is deployed on edge devices (ESP32 or Arduino Nano 33 BLE Sense). Real-time inference is performed using test samples, and the following metrics are measured directly on hardware:

- Accuracy of predicted locations.
- Latency per inference.
- RAM and flash memory usage.
- Energy consumption per inference.

D. Reproducibility

All experiments rely on publicly available datasets and documented preprocessing steps, enabling reproducibility by other researchers or students.

E. Expected Results

Based on prior studies and TinyML optimization techniques, the expected outcomes are summarized in Table IV.

TABLE IV
EXPECTED PERFORMANCE OF THE PROPOSED TINYML SYSTEM

Metric	Expected Range / Target
Localization Accuracy	85–95% (room-level)
Memory Footprint	≤ 500 KB (quantized student model)
RAM Usage	≤ 100 KB during inference
Inference Latency	≤ 100 ms per sample
Energy Consumption	≤ 100 mW per inference
Deployment Feasibility	Fully offline, no cloud dependency

Quantization and knowledge distillation are expected to reduce memory usage and latency significantly while maintaining competitive localization accuracy. The system demonstrates the practical feasibility of TinyML for real-time indoor localization on low-cost, resource-constrained edge devices.

V. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presents a TinyML-based approach for indoor localization on resource-constrained edge devices using publicly available datasets such as UJIndoorLoc. By applying model optimization techniques including knowledge distillation and quantization, the proposed system achieves a compact neural network capable of real-time inference on low-power microcontrollers such as the ESP32 and Arduino Nano 33 BLE Sense.

The methodology ensures reduced memory footprint, low latency, and minimal energy consumption while maintaining competitive localization accuracy. Experimental evaluation on edge devices demonstrates that on-device inference is feasible

without relying on cloud infrastructure, making the system suitable for deployment in cost-sensitive and resource-limited environments.

B. Future Work

Future extensions of this work include:

- **Extended Datasets:** Incorporating additional public datasets or crowdsourced data to improve generalization across diverse building layouts.
- **Hybrid Signal Modalities:** Combining Wi-Fi RSSI with BLE, magnetic, or inertial measurements for higher localization accuracy.
- **Advanced Model Compression:** Exploring pruning, low-rank factorization, or neural architecture search to further reduce model size while preserving performance.
- **Adaptive Edge Deployment:** Implementing dynamic model switching or on-device retraining for real-time adaptation to changing signal environments.
- **Deployment in Real-World Scenarios:** Testing the system in operational edge computing applications such as robotics, augmented reality, and industrial monitoring.

These extensions aim to enhance the robustness, scalability, and practical usability of TinyML-based indoor localization systems in real-world edge computing scenarios.

REFERENCES

- [1] Suwannaphong, T., Jovan, F., Craddock, I. et al. Optimising TinyML with quantization and distillation of transformer and mamba models for indoor localisation on edge devices. *Sci Rep* 15, 10081 (2025). <https://doi.org/10.1038/s41598-025-94205-9>
- [2] Ishana Attri, Lalit Kumar Awasthi, T.P Sharma, TinyML for Plant Disease Detection: Efficient Edge AI Solutions for Apple and Mango Leaves, *Procedia Computer Science*, Volume 258, 2025, Pages 2870-2877, ISSN 1877-0509.
- [3] Zeynali, M., Alipour, K., Tarvirdizadeh, B. et al. Non-invasive blood glucose monitoring using PPG signals with various deep learning models and implementation using TinyML. *Sci Rep* 15, 581 (2025). <https://doi.org/10.1038/s41598-024-84265-8>
- [4] Abadade, Y. et al. A comprehensive survey on tinyml. *IEEE Access* (2023).
- [5] Polino, A., Pascanu, R. Alistarh, D. Model compression via distillation and quantization. *arXiv preprint arXiv:1802.05668* (2018). [6]